

# CSC111 Project 2 Final Report

## Designing a Simple AI for a Simplified Version of Blokus Using Game Trees

Arian Mahlooji  
Ishaan Kelkar  
Hoang Gia Bach

April 2, 2026

## 1 Introduction

Blokus is a strategic board game in which players place pieces on a grid under strict placement constraints. Each piece must touch another piece of the same color at a corner, and pieces of the same color cannot share edges. The objective is to maximize the number of squares occupied while limiting the opponent's options.

In this project, we implemented a simplified version of Blokus on a  $6 \times 6$  board to make the problem computationally feasible while preserving its strategic depth.

### **Research Question:**

**How can we design an AI using a game tree for a simplified version of Blokus, and how effective is this AI across different levels of difficulty compared to simpler strategies such as random and greedy players?**

## 2 Dataset

This project does not rely on external datasets. Instead, all data is generated through simulation.

Each simulated game produces:

- the winner
- the number of turns
- the final scores of both players

We ran multiple batches of experiments (each consisting of 20 games per difficulty level), allowing us to generate a dataset large enough to analyze trends in win rates, scores, and gameplay behavior.

## 3 Computational Overview

### 3.1 Game Representation

The core of the program is the GameState class in `blokus_state.py`.

It represents:

- the board (2D list)
- the current player
- remaining pieces
- pass count

Key methods include:

- `get_legal_moves`, which generates all valid moves
- `is_legal_placement`, which enforces the rules of the game
- `apply_move`, which produces new game states

Moves are represented using a `Move` data class. To ensure correctness and avoid unintended side effects, the class is defined using:

```
@dataclass(frozen=True)
```

This makes each `Move` object immutable after creation. Since moves represent fixed actions in the game, immutability prevents accidental modification and ensures that they can be safely used across different game states and tree nodes. This design choice improves reliability and aligns with best practices for representing value objects.

## 3.2 Game Tree and Algorithm

The game is modeled using the `GameTree` class in `blokus.tree.py`.

Each node represents a game state

Each edge represents a move

The tree is generated using `generate_tree(depth)`

This function:

- Calls `GameState.get_legal_moves` to generate moves
- Recursively builds child states
- Assigns evaluation values to nodes

A minimax-style approach is used:

- the RED player maximizes the evaluation
- the BLUE player minimizes it

## 3.3 Evaluation Function

The evaluation function is defined in `evaluate_state`:

$$2.0 \times (\text{score difference}) + 0.5 \times (\text{mobility difference})$$

This combines:

- board control (score)
- future flexibility (mobility)

### 3.4 Player Strategies and Difficulty Modes

Player strategies are implemented in `players.py`:

- `RandomPlayer`: selects a move randomly
- `GreedyPlayer`: selects the move with the best immediate evaluation
- `TreeAIPlayer`: builds a game tree and selects moves using minimax

To extend the project beyond the original proposal, we introduced a difficulty mode system:

- Easy difficulty uses `RandomPlayer`
- Medium difficulty uses `GreedyPlayer`
- Hard difficulty uses `TreeAIPlayer`

This creates a clear progression in AI strength and adds an extra layer of customization and challenge to the project.

### 3.5 Simulation

Simulation is implemented in `simulation.py`:

- `run_game` simulates a single game
- `run_many_games` runs multiple games
- `summarize_results` computes statistics

### 3.6 Visualization (Plotly)

Visualization is implemented in `visualization.py` using the Plotly library.

The implementation uses:

- `plotly.graph_objects`
- `go.Figure` to construct charts
- `go.Bar` to create bar graphs
- `fig.show()` to display results

The program generates:

- win rate graphs
- average score graphs

These visualizations allow for clear comparison between strategies and difficulty levels.

### 3.7 Implementation Detail: Mutable State Handling

In the `GameTree` class, children are defined as:

```
children: list[GameTree] = field(default_factory=list)
```

This ensures that each node has its own independent list of children. Without `default_factory`, Python would reuse the same list across instances, which would corrupt the structure of the game tree.

## 4 Instructions for Running the Program

Install dependencies:

```
pip install -r requirements.txt
```

Run the program:

```
python main.py
```

The program will:

- print results in the terminal
- display interactive graphs in a web browser

## 5 Changes from Proposal

Compared to the initial proposal, several changes were made:

- Introduced a difficulty mode system (easy, medium, hard)
- Added a GreedyPlayer as an intermediate strategy
- Limited tree depth to ensure reasonable runtime
- Focused on simulation and visualization instead of building a full user interface

## 6 Additional Features and Extensions

This project extends beyond a basic implementation of a game by functioning as a computational experiment platform.

The addition of difficulty modes allows the system to represent different levels of decision-making, from random behavior to deeper strategic reasoning. This makes the system more flexible and allows for structured comparisons between AI strategies.

In addition, the simulation framework enables repeated experiments across many games, allowing statistical analysis rather than relying on single outcomes. The use of Plotly further enhances this by presenting results visually, making comparisons clearer and more intuitive.

## 7 Discussion

To evaluate the effectiveness of our AI strategies and difficulty modes, we conducted multiple experiments by running 20 simulated games for each difficulty level against a RandomPlayer baseline. The results provide clear evidence of how increasing the sophistication of decision-making improves performance in the game.

For the easy difficulty, which uses the RandomPlayer strategy, the results show relatively weak performance. In one experiment, the easy player achieved a red win rate of 0.35, a blue win rate of 0.10, and a tie rate of 0.55, with an average red score of 8.8 and an average blue score of approximately 7.6–7.85. These results are expected, as both players are selecting moves randomly, leading to highly unpredictable outcomes and a large number of ties. The relatively high tie rate indicates that neither player consistently gains a strong advantage, and the game outcomes are largely influenced by chance rather than strategy. Because both players behave randomly, the system lacks any consistent pattern of improvement, and performance remains unstable across different runs.

For the medium difficulty, which uses the GreedyPlayer strategy, the performance improves slightly. In our experiments, the red win rate ranged between 0.35 and 0.50, with no blue wins observed and tie rates between 0.50 and 0.65. The average red score remained around 9.0, while the average blue score dropped to approximately 6.1–7.7. This demonstrates that even a simple heuristic-based strategy can outperform purely random play. The greedy player evaluates moves based on immediate board advantage, which allows it to consistently secure slightly better positions compared to a random player. However, the high tie rates suggest that the greedy strategy is still limited, as it does not consider future consequences and can make locally optimal but globally suboptimal decisions. As a result, while it improves average performance, it does not consistently dominate the opponent.

The most significant improvement is observed in the hard difficulty, which uses the TreeAIPlayer with a game tree and minimax-style evaluation. In one experiment, the hard player achieved a red win rate of 0.75, with no blue wins and a tie rate of 0.25. In another run, the win rate remained strong at 0.55, again with zero losses and a tie rate of 0.45. The average red score consistently remained around 9.0, while the average blue score dropped significantly to approximately 5.45–6.4. These results clearly demonstrate the advantage of using a game tree to evaluate future states. Unlike the greedy strategy, the TreeAIPlayer considers multiple possible future sequences of moves, allowing it to avoid short-term gains that lead to poor long-term outcomes. This forward-looking approach leads to more consistent and stronger performance across different simulations.

Another important observation is the change in the average number of turns. Across all difficulty levels, the average number of turns remained relatively stable, ranging between 7.25 and 7.65. This suggests that stronger strategies do not necessarily prolong the game but instead improve efficiency in reaching advantageous positions. In fact, the slightly lower average number of turns observed in the hard difficulty indicates that the TreeAIPlayer is able to reach decisive outcomes more quickly by making better decisions earlier in the game. This highlights that improved intelligence in the AI leads not only to better outcomes but also to more efficient gameplay.

Overall, the results strongly support our research question. As the difficulty level increases from easy to hard, the AI becomes significantly more effective. The transition from random to greedy introduces basic strategic reasoning, while the transition to a tree-based approach introduces forward planning and deeper analysis. This progression clearly shows that considering future states is critical in decision-making problems such as this game. The difficulty modes successfully capture different levels of intelligence and provide a meaningful way to compare strategies.

Despite these strengths, there are several limitations in our approach. The depth of the game tree is limited due to the exponential growth of possible states, which restricts how far ahead the AI can search. Additionally, the evaluation function is relatively simple and may not capture all aspects of strategic gameplay, such as long-term positioning or blocking strategies. Furthermore, although the difficulty modes provide different levels of challenge, the system does not currently include a human-interactive mode, which would further enhance usability and engagement.

In future work, the project could be improved by implementing optimization techniques such as alpha-beta pruning to reduce the number of states evaluated, allowing deeper searches. The evaluation function could also be refined to incorporate more complex strategic features. Finally, adding an interactive mode where a human player can compete against different difficulty levels would significantly extend the project and make it more practical and more engaging.

## References

- [1] "Blokus Official Rules." *Mattel Games*,  
<https://www.mattelgames.com/en-us/strategy/blokus>.
- [2] "Plotly Python Graphing Library Documentation." *Plotly*,  
<https://plotly.com/python/>.
- [3] Russell, Stuart, and Peter Norvig.  
*Artificial Intelligence: A Modern Approach*.  
Pearson Education, 2010.
- [4] OpenAI. "ChatGPT." *OpenAI*, 2024,  
<https://chatgpt.com/>.